# Computer-Orchestrated Design of Algorithms: From Join Specification to Implementation

Zeyuan Hu
University of California, Los Angeles
Los Angeles, California, USA
zeyuanhu@cs.ucla.edu

## Abstract

Equipping query processing systems with provable theoretical guarantees has been a central focus at the intersection of database theory and systems in recent years. However, the inevitable divergence between theoretical abstractions and system assumptions creates a significant gap between an algorithm's high-level logical specification and its low-level physical implementation. Ensuring the correctness of this logical-to-physical translation is crucial for realizing theoretical optimality as practical performance gains. Existing database testing frameworks struggle to address this need because necessary algorithm-specific inputs such as join trees (as opposed to query plans) are absent from standard test case generation, and integrating complex algorithms into these frameworks imposes prohibitive engineering overhead. Fallback solutions, such as using macro-benchmark queries, are inherently too noisy for isolating intricate defects during this translation.

In this experience paper, we present a retrospective analysis of CODA, a computer-orchestrated testing framework utilized during the physical co-design of TreeTracker Join (TTJ), a theoretically optimal yet highly practical join algorithm recently published in ACM TODS. By synthesizing minimal reproducible examples, CODA successfully isolates subtle translation defects, such as state mismanagement and intricate mapping conflicts between join trees and bushy plans. We demonstrate that this logical-to-physical translation process is a bidirectional feedback loop: early structural testing not only hardened TTJ's physical implementation but also exposed a critical boundary condition that directly refined the formal precondition of TTJ itself. Finally, we detail how confronting these structural translation challenges drove the architectural evolution of CODA into a highly robust, structure-aware test generation pipeline for join-tree-dependent algorithms.

**ACM Reference Format:**
Zeyuan Hu. 2026. Computer-Orchestrated Design of Algorithms: From Join Specification to Implementation. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX).* ACM, New York, NY, USA, 9 pages. https://doi.org/XXXXXXX.XXXXXXX

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
*Conference acronym 'XX, Woodstock, NY*
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2026/06
https://doi.org/XXXXXXX.XXXXXXX

## 1 Introduction

Query processing remains the cornerstone of any relational database management system (DBMS). In recent years, a major research thrust has emerged to equip practical query evaluation with provable theoretical guarantees. At the heart of this effort are conjunctive queries (CQs), queries involving multiple joins. An important subclass of CQs is acyclic conjunctive queries (ACQs) due to their immense abundance in real-life workloads [3, 10]. This line of research aims to bridge the gap between theory and practice in ACQ evaluation by introducing algorithms that are efficient in both domains (see survey [8]). Notably, this investigation is no longer merely an academic exercise; such theoretical advancements for ACQs have proven their practical viability in production-grade systems, being successfully deployed in DuckDB [11] and independently realized within Microsoft SQL Server's optimization infrastructure [19]. The latter shows a promising direction: re-evaluating mature query optimization heuristics through formal lenses, such as ACQ evaluation. One significant challenge in designing practical ACQ evaluation algorithms is that system constraints commonly diverge significantly from theoretical assumptions. Hence, a gap exists between the high-level logical specification of an algorithm and its low-level physical implementation. Ensuring a faithful logical-to-physical translation is therefore crucial to guarantee that theoretical optimality actually translates into practical performance gains.

A natural choice to verify the physical translation of such algorithms is to leverage conventional database testing frameworks [6, 7, 9, 12–15, 18, 21]. However, incompatibilities exist with the needs of the translation. First, algorithms rely on an input data structure called a join tree [1] that is distinct from a query plan and is not commonly seen in mainstream databases. This fact imposes a significant challenge to conventional database testing frameworks, as the correctness of query evaluation depends not only on the query plans but also on the topology of join trees. However, the existing testing frameworks do not support join trees, and retrofitting them is highly non-trivial. Second, existing frameworks generate SQL queries and database instances with the goal of testing fully-featured DBMSs. Thus, to utilize these tools, researchers would first have to fully integrate their nascent algorithm into an existing DBMS, which imposes an unnecessary engineering burden when the primary goal is simply to verify the correctness of the logical-to-physical translation. Hence, it is natural for researchers to use benchmark queries as a proxy to test their physical translation of an algorithm specification on an experimental prototype that closely mimics system constraints. They have to manually craft a

[1] It is an unfortunate coincidence that the term "join tree" from database theory [1] may also refer to a query plan in DBMS literature. In this paper, we make a distinction between join tree and query plan.

join tree for a given query and check the correctness of the translation through evaluation. The issue with this approach is due to the complexity of benchmarks. Since the purpose of benchmark queries is to mimic real-life workloads and benchmark DBMS performance, those queries, along with the associated database instance, are inherently complex. When a defect is flagged, the sheer size and structural noise of the benchmark query prevent researchers from effectively isolating the root cause.

In this paper, we detail the Computer-Orchestrated Design of Algorithms (CODA)[2] framework utilized during the design of Tree-Tracker Join (TTJ), an optimal yet practical join algorithm recently published in ACM Transactions on Database Systems (TODS) [5]. The term "computer-orchestrated" emphasizes the framework's researcher-centric perspective: instead of black-box bug discovery, CODA explicitly orchestrates test generation based on structural properties of join trees to assist researchers in verifying the intricate logical-to-physical translation. CODA operates directly on join trees, verifying the correctness of the physical implementation by exploiting the interaction between join trees and query plans. Furthermore, CODA proactively exposes defects via *minimal reproducible examples* (MREs), allowing us to rapidly identify subtle translation errors easily masked by benchmark proxies. Notably, the core components of CODA are immediately applicable to a broader class of join-tree-dependent algorithms [8].

Importantly, this retrospective analysis reveals that orchestrating algorithm design creates a bidirectional feedback loop: the defects exposed by CODA did not merely debug TTJ's physical implementation; they fundamentally refined the theoretical precondition of TTJ itself. Simultaneously, tackling these complex translation challenges drove the evolution of CODA itself, culminating in a robust, structure-aware testing framework.

## 2 From Logical to Physical

TTJ evaluates an ACQ such as $Q = R(a, x) \bowtie S(a, w) \bowtie T(a, z)$ for relations $R$, $S$, and $T$ optimally. While its logical specification is a simple 13-line recursive function relying on implicit call-stack state (Figure 2), to ensure an apples-to-apples performance comparison against mature systems like PostgreSQL, TTJ must be implemented as a physical iterator within a standard pipelined execution engine (Algorithm 1, adapted from [4]). This logical-to-physical translation is highly non-trivial: it demands explicit mutual recursion, state tracking, and complex backtracking, essentially doubling the length of its recursive counterpart. The key idea of TTJ is to leverage a data structure called a join tree (different from query plan; we will explain in Section 2.2) to avoid redundant computation and guarantee the optimality. Due to space constraints, we defer step-by-step query evaluation examples of Figure 2 to [5], and the full formal proofs of TTJ iterator's properties to [4].

**Algorithm 1:** Physical TTJ implementation featuring explicit stack and state management alongside mutual recursion between `getNext()` and `deleteDT(R)`.

**Purpose:** An iterator returns, one at a time, the join result of `Router` and `Rinner`.
**Output:** A tuple from `Router` ⋈ `Rinner`

```
TTJOperator
  void open()
    Initialize r, MT to None
    Rinner.open()
    Build hash table inner from Rinner
    Router.open()
  Tuple getNext()
    if MT is not None and MT is not [] then
      if m = MT.next() and m is not None then
        return r ++ m
      r = Router.getNext()
      if r is None then return None
    if r is None || MT = [] then r =
     Router.getNext()
    while r is not None do
      // Find tuples from Rinner joinable with r
      MT =
       inner.get(keys(plan[1..num(this)],Rinner))
      if MT is not None then
        m = MT.next()
        return r ++ m
      else
        // Backjump to parent(Rinner)
        r = Router.deleteDT(parent(Rinner))
    return None
  Tuple deleteDT(Relation R)
    if Rinner == R then
      Remove m from MT and inner
    else
      // Has not reached R; backjumping continues
      MT = None
      r = Router.deleteDT(R)
      if r is None then return None
    return this.getNext()
```

### 2.1 Preliminaries

A TTJ *left-deep plan* is a binary tree where internal nodes are TTJ iterators (Algorithm 1), and every right child `Rinner` and the left child `Router` of the leftmost iterator are leaves; all other plans are *bushy*. Leaves serve as both nodes and relations, indexed left-to-right as $\ell_1, \ldots, \ell_{|Q|}$, with $|Q|$ denoting the number of relations and `plan[1..i]` denoting the prefix sequence $(\ell_1, \ldots, \ell_i)$. Function `keys(plan[1..i],R)` computes R's *key*: the attributes it shares with $\ell_1, \ldots, \ell_{i-1}$. We define `inner` as the hash table built on `Rinner`, and ++ as schema-resolving tuple concatenation. For an iterator with left child $\ell_i$, its label is $i$. Finally, `this` self-references the iterator, and `parent(R)` identifies R's parent in the join tree. We use $\bowtie^T$ to denote a TTJ iterator instance.

[2]In music, a coda is a concluding passage added to the end of a musical piece, providing a sense of finality, closure, or resolution. Similarly, we believe the design of any practical algorithm is truly concluded only when it is rigorously verified and successfully deployed in practice.

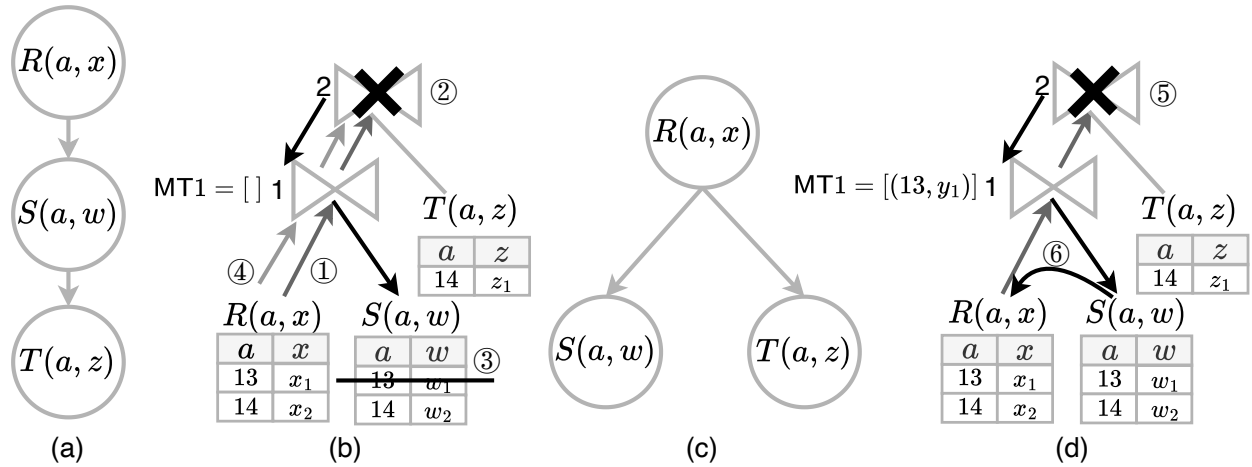


**Figure 1: Illustration of how the evaluation correctness of $Q = R(a, x) \bowtie S(a, w) \bowtie T(a, z)$ depends on the join tree structure. Using a defective TTJ iterator (missing Line 26 in Algorithm 1), $Q$ evaluates correctly under the join tree in (a), as traced in (b). Conversely, under the join tree in (c), the identical defective iterator yields an incorrect empty result, as traced in (d). We explain the details in Section 2.2. Symbols ①through ⑥mark execution snapshots referenced in the text.**

```
def ttj(t, plan, i):
  if i > plan.len(): print(t)
  else:
    R = plan[i]; k = π_{keys(plan[1..i],R)}(t)
    P = parent(plan[1..i], R)
    if R[k] is None & P is not None:
      return P # backjump to P
    for r in R[k]:
      result = ttj(t++r, plan, i+1)
      if result == R: # catch backjump
        R[k].delete(r)
      elif result is not None:
        return result # continue backjump
```

**Figure 2: Logical TTJ specification from [5], featuring the simplicity of implicit state management via the call stack.**

## 2.2 A Motivating Example

Consider the ACQ $Q$ defined in Section 2 evaluated over the database instance in Figure 1, where $x_i, w_i, z_j$ ($i \in \{1, 2\}$ and $j = 1$) denote distinct constants. $Q$ is acyclic if it admits a join tree. A *join tree* is a tree where there is a one-to-one correspondence between nodes in the tree and relations in the query; two nodes are adjacent if and only if their corresponding relations share common attribute(s); and the tree must satisfy the *running intersection property* (RIP): for any attribute $a$ that appears in $Q$, all relations containing $a$ form a connected subtree in the join tree. Once rooted, the directed join tree dictates that execution must backjump along the tree edges, rather than the query plan, upon a probe failure. Multiple join trees can exist for a given query; for example, Figure 1 (a) and (c) illustrate two valid join trees for $Q$, differing in their parent mappings where `parent(T)` is $S$ in tree (a), but $R$ in tree (c).

Consider a defective TTJ iterator missing Line 26, which is responsible for clearing the local state MT during a backjump. Testing this defective iterator under join tree (a) falsely reports success, while tree (c) correctly reports output inconsistency, exposing the flaw.

Suppose we evaluate $Q$ under join tree (a) using the plan from Figure 1 (b). Following the pipelined execution, the iterator $\bowtie_1^T$ fetches $R(13, x_1)$, successfully probes $S$, and caches the match $S(13, w_1)$ in MT (①). When the subsequent probe against $T$ fails (②), Line 20 from $\bowtie_2^T$ is invoked, and the execution backjumps to $\bowtie_1^T$ due to its right child $S$ being the parent of $T$. This correctly invokes Line 24, flushing the stale state in MT (③). The iterator $\bowtie_1^T$ then fetches $R(14, x_2)$, probes both hash tables successfully (④), and yields the correct output.

Conversely, evaluating $Q$ under join tree (c) using the plan from Figure 1 (d) exposes the defect. Upon the same probe failure at $T$ (⑤), iterator $\bowtie_2^T$ calls $\bowtie_1^T$`.deleteDT(R)` via Line 20. Because the defective iterator lacks Line 26, it skips clearing the state for the bypassed relation $S$. Instead, $\bowtie_1^T$ directly calls `R.deleteDT(R)` (which simply invokes `getNext()` of $R$) from Line 27 (⑥). Consequently, MT incorrectly retains the stale tuple $S(13, w_1)$. When $R(14, x_2)$ is fetched, the iterator $\bowtie_1^T$ processes this corrupted state, fails its validity check at Line 9 (since $S(13, w_1)$ has already been processed), and erroneously terminates with an empty result via Line 12.

Comparing these traces reveals a critical pitfall: a flawed physical translation can easily evade detection and produce correct results if the testing benchmark happens to utilize a forgiving topology like the degenerate tree (a). Relying solely on benchmark queries leaves the detection of such state-corruption defects to chance.

This demonstrates the absolute necessity of structure-aware testing by manipulating join tree topologies directly during the early translation stage to isolate exact root causes.

## 3 CODA Framework

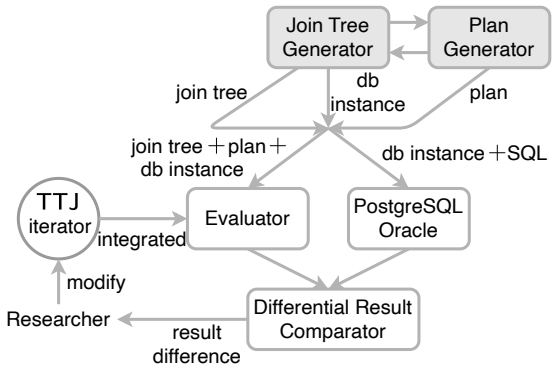


**Figure 3: Overview of the CODA framework used during the design of the TTJ iterator (Algorithm 1). Test case synthesis modules are gray rounded rectangles, while differential evaluation modules are white rounded rectangles. The TTJ iterator implementation is depicted as a circle.**

Figure 3 illustrates the architecture of CODA utilized during the design of the TTJ iterator. While implemented for TTJ, CODA is highly modular: its algorithm-agnostic components readily support other join-tree algorithms by simply replacing the TTJ iterator in the figure. CODA operates in two tightly coupled phases: test case synthesis and differential evaluation.

To isolate defects efficiently, CODA synthesizes MREs rather than massive benchmark workloads, capping query size at `max_size` = 5 relations and relation size at `max_rel_size` = 10 tuples. Notably, this small configuration is sufficient to expose the defects we highlight in Section 4. The synthesis pipeline consists of the join tree generator and the plan generator, either of which can be used to initiate the synthesis process. The plan generator synthesizes a collection of relations and applies random permutations and parenthesizations to construct arbitrary left-deep or bushy query plans. Figure 4 illustrates the functionality of the join tree generator when generating a query expressed directly as a join tree. The `generateRandomTree` function first builds a random directed tree. Then, the `generate` function enforces RIP by utilizing a top-down breadth-first traversal: each child node restricts its attribute pool to a random subset of its parent's schema (Line 26). This structural inheritance guarantees that the resulting tree inherently satisfies RIP and hence is a valid join tree. Relation instances are populated alongside the join tree construction and subsequently ingested into the PostgreSQL oracle.

During evaluation, CODA translates the ACQ's logical semantics into a standard `SELECT-FROM-WHERE` SQL query to obtain the ground truth from PostgreSQL. Enforcing a specific join-tree topology is unnecessary, as PostgreSQL's internal execution plan is

```
1  def generateRandomTree(n):
2    nodes = [Node() for _ in range(n)]
3    T = Tree()
4    root = random_choice(nodes)
5    S = [root]; nodes.remove(root)
6    while nodes:
7      u = random_choice(nodes)
8      v = random_choice(S)
9      T.addEdge(v, u); S.append(u); nodes.remove(u)
10   return (T, root)
11
12 def generate(max_size, max_rel_size, attr_pool):
13   T, root = generateRandomTree(random(1, max_size))
14   inherited = {root: attr_pool}
15   x = random(1, len(attr_pool))
16   queue = [root]
17   while queue:
18     next_level = []
19     for node in queue:
20       # sample x attrs from inherited pool, enforcing RIP
21       schema = sample_nonempty(inherited[node], x)
22       tuples = gen_tuples(schema, random(1, max_rel_size))
23       postgres.ingest(Relation(node.name, schema), tuples)
24       for child in T.children(node):
25         # child inherits parent schema
26         inherited[child] = schema
27         next_level.append(child)
28     # narrow pool after each BFS level
29     x = max(x - random(0, x), 1)
30     queue = next_level
31   return T
```

**Figure 4: Synthesizing a join tree and a database instance in CODA in the absence of an input query plan.**

strictly orthogonal to the logical result. For exact output equivalence, synthesized relations exclude `NULL`s, and the SQL uses bag semantics to match TTJ's multiset output. Simultaneously, CODA binds TTJ iterators to the query plan. A differential comparator then cross-verifies outputs to flag isolated defects.

Initially, CODA fed plans directly to the evaluator, forcing TTJ to map the plan to a default join tree at runtime. As demonstrated in Sections 4.2 and 4.3, this direct coupling led to semantic collapses. This forced CODA into its current architecture (Figure 3), where the join tree generator acts as a mandatory validation layer (code omitted for brevity). Furthermore, when the generator is tasked with constructing a valid join tree from an arbitrary bushy plan, the structural translation becomes highly non-trivial. We defer the detailed explanation of this complex procedure to Sections 4.2 and 4.3, where we contextualize it alongside the specific logical-to-physical translation defects it resolved.

## 4 Evaluation

In this section, we demonstrate the critical role of CODA in the physical translation of TTJ through three postmortem case studies.

**Table 1: Diagnostic utility comparison between a standard macro-benchmark (TPC-H) and the CODA framework across critical logical-to-physical translation defects. Note: TPC-H counts represent queries, whereas CODA counts represent MREs.**

| Defect Class | TPC-H | CODA Suite | Limitation of Macro-Benchmarks |
|---|---|---|---|
| **1. State Mismanagement** | Detected (4 queries) | Isolated (8 MREs) | Requires a specific combination of non-degenerate join tree topologies and adversarial data distributions to trigger. |
| **2. GYO Reduction Order Violation** | Missed (0 queries) | Isolated (1 MRE) | Benchmark query plans are structurally homogeneous (easily mapped to join trees), failing to stress-test TTJ's theoretical precondition. |
| **3. RIP Violation** | Detected (2 queries) | Isolated (1 MRE) | Complex bushy plans from benchmark queries introduce significant structural noise, severely hindering precise defect isolation. |

These defects fundamentally shaped the theoretical boundaries of TTJ, its physical iterator implementation, and the architectural evolution of CODA itself. Overall, CODA synthesized 28 minimal test cases that cover a wide range of defects discovered during the TTJ development lifecycle; Table 1 highlights 10 of these cases categorized into three representative defect classes, while the remaining 18 test cases cover additional variants and edge cases. The CODA suite is released along with the source code of the TTJ query engine (implemented in Java); comprehensive reproducibility instructions, including hardware specifications and the PostgreSQL version (v13), can be found in our previous work [5]. To illustrate the limitations of relying on macro-benchmarks as a testing proxy, and to demonstrate how CODA systematically addresses these blind spots, we benchmark our findings against the same 13 standard TPC-H multi-join queries [16] (Q3, Q7–Q12, Q14–Q16, Q18–Q20) as we did in [5]. Given the retrospective nature of this paper, we defer a more rigorous comparison with sophisticated database testing frameworks to future work.

Table 1 summarizes the comparison. TPC-H counts represent complex, end-to-end benchmark queries, whereas CODA isolates defects using MREs. Overall, CODA synthesized 28 failing cases across the entire development lifecycle of TTJ. For this evaluation, we distilled these into 10 distinct MREs by filtering out transient defects associated with early prototypes and categorizing the remaining core translation defects by their underlying root causes. The following subsections detail three representative MREs. The first defect (Section 4.1) demonstrates that exposing intricate state corruptions requires a precise combination of join tree topology and database instance. While TPC-H occasionally triggers the defect, there is no clean topological categorization between queries that mask the issue and those that expose it. This ambiguity forces researchers to dive into the massive, noisy database instance to isolate the root cause. In contrast, CODA pinpoints the exact failure using merely three relations and five tuples. The second defect (Section 4.2) exposes a deeply hidden theoretical limitation of TTJ that TPC-H entirely missed, which ultimately forced us to refine the formal precondition of TTJ. The third defect (Section 4.3) highlights a structural translation hazard. While TPC-H eventually surfaced the defect using sprawling 6- or 8-relation bushy plans, CODA achieved topological isolation by reproducing the identical defect using a minimal structure of just three relations.

## 4.1 Case Study 1: State Mismanagement

This case study revisits the state mismanagement introduced in Section 2.2, caused by the omission of Line 26 from Algorithm 1. While TPC-H benchmark queries eventually surface the defect, they offer poor diagnostic utility. The defect was flagged by only four queries (Q3, Q7, Q8, and Q10), all of which possess non-degenerate join trees (i.e., at least one internal node has multiple children, akin to Figure 1 (c)). The defect remained completely dormant in the remaining nine queries. Unsurprisingly, the defective TTJ iterator masked the defect across the seven queries featuring degenerate join trees (akin to Figure 1 (a)). However, the defect also remained hidden in Q9 and Q11 despite their non-degenerate topologies. As a result, there is no clean topological categorization between the TPC-H queries that mask the defect and those that expose it.

The above observation suggests that topological structure alone is insufficient to trigger the defect; it requires specific data distributions. The inconsistent coverage of TPC-H benchmark queries forces researchers to sift through the massive, noisy database instance hoping for coincidental data alignment. By contrast, the value of CODA lies not just in detection, but in immediate isolation: it deterministically synthesizes a minimal MRE consisting of just three relations and five tuples (Section 2.2), bypassing the overhead of macro-benchmark noise.

## 4.2 Case Study 2: Refining the Theoretical Precondition

The next two case studies concern the physical translation of the `parent(Rinner)` logic (Line 20 of Algorithm 1). To achieve its theoretical guarantees, the iterator must dynamically recover a valid join tree from the provided plan. Let $\Sigma(R)$ denote the attribute set of relation $R$. The recovery procedure computes the key $K$ of `Rinner` and assigns `Rinner`'s parent to $\ell_j$, where $j$ is $\min\{q \in \{1, \ldots, i-1\} \mid K \subseteq \Sigma(\ell_q)\}$. This produces the shallowest possible join tree, maximizing backjumping distance on probe failures and pruning redundant intermediate results.

Initially, CODA sent left-deep plans directly to the evaluator under the naive assumption that mapping any ACQ left-deep plan to a join tree was trivial and could thus be performed during the evaluation phase. This assumption catastrophically failed when CODA synthesized a simple left-deep plan $P = R(a, b) \bowtie S(b, c) \bowtie T(a, b, c)$, instantly triggering a `NullPointerException` at Line 20.

While $P$ is undeniably an ACQ (admitting a valid join tree rooted at $T$), the recovery failed because the key of $T$ is $\{a, b, c\}$, yet neither $\Sigma(R)$ nor $\Sigma(S)$ contains this set. This defect directly refined our initial theoretical conjectures about TTJ.[3] We originally assumed TTJ guaranteed linear-time evaluation on arbitrary left-deep plans. Confronted with CODA's counterexample (and corroborated by formal review), we restricted TTJ's optimality guarantee [5] strictly to left-deep plans satisfying the reverse of a GYO reduction order.

Relying solely on macro-benchmarks would have created the illusion that optimality applies to arbitrary left-deep plans, leaving this theoretical flaw permanently undiscovered. A follow-up study [10] proved that across 10,454 queries in five popular benchmarks (including TPC-H), every single left-deep plan inadvertently satisfies the reverse of a GYO reduction order. This structural uniformity inherently masks such theoretical boundary conditions.

### 4.3 Case Study 3: Architectural Evolution and Topological Isolation

Following the discovery of the GYO reduction order constraint (Section 4.2), we initially attempted a lightweight patch by adding an inline validation check within `parent(Rinner)`. However, this check consistently raised false alarms when CODA injected valid ACQ bushy plans into the evaluator.

To understand the topological mapping failure, consider two bushy plans derived from the same relations: $P_1 = R(a, b) \bowtie (T(a, b, c) \bowtie S(b, c))$ and $P_2 = T(a, b, c) \bowtie (R(a, b) \bowtie S(b, c))$. For $P_1$, recursively applying the runtime recovery method from Section 4.2 operates bottom-up: it first builds a subtree $T \rightarrow S$ for the inner subplan $T \bowtie S$, and then connects the outer relation $R$ to the subtree root $T$, correctly yielding the degenerate join tree $R \rightarrow T \rightarrow S$. However, applying the identical logic to $P_2$ first builds a subtree $R \rightarrow S$ for $R \bowtie S$, and then connects the outer relation $T$ to the root $R$, yielding a tree $T \rightarrow R \rightarrow S$. This topology is not a join tree because it violates RIP: $T$ and $S$ share attribute $c$, but are disconnected by $R$.

To maintain a unified execution logic of the evaluator that maps any query plan to a single valid join tree, we introduce the abstraction of *virtual relations* to restore RIP in the constructed tree for a bushy plan like $P_2$. Instead of directly connecting the outer relation $T$ with $R$, we abstract the $R \bowtie S$ subplan as a virtual relation $V(a, b, c)$ where its attribute set $\Sigma(V)$ is the union of $\Sigma(R)$ and $\Sigma(S)$. The resulting valid join tree roots at $T(a, b, c)$, connected to $V(a, b, c)$, which in turn has two children $R(a, b)$ and $S(b, c)$. This virtual relation abstraction successfully covers all 139 bushy plans in our benchmarks [5]. (A formal characterization of when this approach is applicable is available in Appendix A.)

Building on the breakdown of the trivial runtime mapping assumption (Section 4.2), the complexity of virtual relations proved that in-iterator join tree recovery is architecturally unsustainable. This realization triggered the architectural evolution of CODA to its final state (Figure 3), entirely offloading the responsibilities of GYO validation (Section 4.2) and join tree construction to CODA's synthesis pipeline. While TPC-H Q8 (an 8-relation join) and Q9 (a 6-relation join) eventually triggered this RIP violation, their sprawling plans were unnecessarily complicated, requiring significant effort to trace the structural root cause. By contrast, CODA reproduced the identical defect using a minimal structure of just three relations.

[3]Remarkably, this defect was concurrently identified by a referee for our ICDT '25 submission.

## 5 Related Work

Extensive research applies differential and generator-based testing to validate database implementations [6, 7, 12–15, 21], typically executing synthesized SQL queries against mature DBMSs. While sharing this differential testing spirit, CODA differentiates itself in three ways. (1) Agility for nascent algorithms: unlike existing tools that assume stable architectures, CODA is lightweight, avoiding the prohibitive overhead of integrating highly volatile algorithm prototypes (e.g., TTJ) into mature systems. (2) Micro-topological synthesis: instead of operating at the macroscopic SQL interface and relying on query optimizers whose heuristics may miss problematic plans and join trees [5, 10, 20], CODA directly synthesizes join trees and query plans, allowing it to stress-test both physical implementations and theoretical preconditions. (3) Topological isolation: by focusing on join trees, CODA implicitly categorizes multi-join queries based on acyclicity, enabling minimal-by-construction test cases for precise defect isolation. This focus on minimality reflects a pragmatic design choice: CODA does not aim to completely eliminate logical-to-physical translation errors; it simply aims to serve as a first-line filter for structural defects. Hence, unlike earlier work such as QuickCheck [2], CODA heuristically imposes an upper bound on the test case size; any error requiring a test case beyond this bound is left to be discovered during the formal proof stage.

## 6 Concluding Remarks

Our retrospective analysis of CODA demonstrates that bridging logical specifications and physical implementations is a bidirectional feedback loop: early structural testing hardened TTJ's complex iterator and directly refined its theoretical precondition. This highlights the danger of relying solely on macro-benchmarks, which actively mask translation defects. As a valuable early-stage vanguard to formal verification, CODA isolates non-trivial defects via minimal reproducible examples long before the prohibitive effort of writing formal proofs. Future extensions include integrating state-of-the-art join tree enumeration [10] to optimize test construction, bridging CODA with mature database testing frameworks to establish a structure-aware testing harness for other join-tree-based algorithms in production-grade systems (e.g., adapting CODA for Yannakakis's algorithm [17] simply requires replacing the TTJ iterator, as core components like the join tree generator remain algorithm-agnostic), and generalizing this structure-aware methodology to the early conceptualization phase of novel algorithms.

## Acknowledgments

The author thanks Daniel Miranker for early guidance on the TTJ project, advisor Remy Wang for helpful discussions, and Yixuan Ni and Zhiting Zhu for manuscript feedback.

## References


[1] Serge Abiteboul, Richard Hull, and Victor Vianu. 1995. *Foundations of Databases*. Vol. 8. Addison-Wesley Reading.

[2] Koen Claessen and John Hughes. 2000. QuickCheck: a lightweight tool for random testing of Haskell programs. In *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming (ICFP '00), Montreal, Canada, September 18-21, 2000*, Martin Odersky and Philip Wadler (Eds.). ACM, 268–279. doi:10.1145/351240.351266

[3] Wolfgang Fischl, Georg Gottlob, Davide Mario Longo, and Reinhard Pichler. 2021. HyperBench: A Benchmark and Tool for Hypergraphs and Empirical Findings. *ACM J. Exp. Algorithmics* 26 (2021), 1.6:1–1.6:40. doi:10.1145/3440015

[4] Zeyuan Hu and Daniel P. Miranker. 2024. TreeTracker Join: Turning the Tide When a Tuple Fails to Join. *CoRR* abs/2403.01631 (2024). arXiv:2403.01631 doi:10.48550/ARXIV.2403.01631

[5] Zeyuan Hu, Yisu Remy Wang, and Daniel P. Miranker. 2026. TreeTracker Join: Simple, Optimal, Fast. *ACM Transactions on Database Systems* 51, 2 (2026), 1–26.

[6] Zu-Ming Jiang and Zhendong Su. 2024. Detecting Logic Bugs in Database Engines via Equivalent Expression Transformation. In *18th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2024, Santa Clara, CA, USA, July 10-12, 2024*, Ada Gavrilovska and Douglas B. Terry (Eds.). USENIX Association, 821–835. https://www.usenix.org/conference/osdi24/presentation/jiang

[7] Jinho Jung, Hong Hu, Joy Arulraj, Taesoo Kim, and Woon-Hak Kang. 2019. APOLLO: Automatic Detection and Diagnosis of Performance Regressions in Database Systems. *Proc. VLDB Endow.* 13, 1 (2019), 57–70. doi:10.14778/3357377.3357382

[8] Paraschos Koutris, Stijn Vansummeren, Qichen Wang, Yisu Remy Wang, and Xiangyao Yu. 2026. Database Theory in Action: Yannakakis' Algorithm. In *Proceedings of 29th International Conference on Database Theory.*

[9] Jinhui Lai, Chi Zhang, Bingyan Li, Chenglin Liang, Jie Liang, Zhiyong Wu, Jingzhou Fu, Yu Jiang, and Zichen Xu. 2025. SRS: Detecting Logic Bugs of Join Implementation in DBMSs via Set Relation Synthesis. *Proc. ACM Manag. Data* 3, 6 (2025), 1–24. doi:10.1145/3769828

[10] Zheng Luo, Wim Van den Broeck, Guy Van den Broeck, and Yisu Remy Wang. 2025. Algorithms for Optimizing Acyclic Queries. *CoRR* abs/2509.14144 (2025). arXiv:2509.14144 doi:10.48550/ARXIV.2509.14144

[11] Yiming Qiao, Peter Boncz, and Huanchen Zhang. 2026. Robust Predicate Transfer with Dynamic Execution. *Proc. VLDB Endow.* 19, 6 (2026), 1278–1290. DuckDB Merged Pull Request: https://github.com/duckdb/duckdb/pull/20633.

[12] Manuel Rigger and Zhendong Su. 2020. Testing Database Engines via Pivoted Query Synthesis. In *14th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2020, Virtual Event, November 4-6, 2020*. USENIX Association, 667–682. https://www.usenix.org/conference/osdi20/presentation/rigger

[13] Andreas Seltenreich, Bo Tang, and Sjoerd Mullender. [n. d.]. SQLSmith. https://github.com/anse1/sqlsmith Accessed: March 2026.

[14] Jiansen Song, Wensheng Dou, Ziyu Cui, Qianwang Dai, Wei Wang, Jun Wei, Hua Zhong, and Tao Huang. 2023. Testing Database Systems via Differential Query Execution. In *45th IEEE/ACM International Conference on Software Engineering, ICSE 2023, Melbourne, Australia, May 14-20, 2023*. IEEE, 2072–2084. doi:10.1109/ICSE48619.2023.00175

[15] Xiu Tang, Sai Wu, Dongxiang Zhang, Feifei Li, and Gang Chen. 2023. Detecting Logic Bugs of Join Optimizations in DBMS. *Proc. ACM Manag. Data* 1, 1 (2023), 55:1–55:26. doi:10.1145/3588909

[16] Transaction Processing Performance Council (TPC). [n. d.]. TPC-H Benchmark. Online. http://tpc.org/tpc_documents_current_versions/pdf/tpc-h_v3.0.0.pdf Accessed: March 2026.

[17] Mihalis Yannakakis. 1981. Algorithms for Acyclic Database Schemes. In *Very Large Data Bases, 7th International Conference, September 9-11, 1981, Cannes, France, Proceedings*. IEEE Computer Society, 82–94.

[18] Chi Zhang and Manuel Rigger. 2025. Constant Optimization Driven Database System Testing. *Proc. ACM Manag. Data* 3, 1 (2025), 24:1–24:24. doi:10.1145/3709674

[19] Hangdong Zhao, Yuanyuan Tian, Rana Alotaibi, Bailu Ding, Nicolas Bruno, Jesús Camacho-Rodríguez, Vassilis Papadimos, Ernesto Cervantes Juárez, César A. Galindo-Legaria, and Carlo Curino. 2026. I Can't Believe It's Not Yannakakis: Pragmatic Bitmap Filters in Microsoft SQL Server. In *16th Conference on Innovative Data Systems Research, CIDR 2026, Chaminade, CA, USA, January 18-21, 2026*. www.cidrdb.org. https://vldb.org/cidrdb/2026/i-cant-believe-its-not-yannakakis-pragmatic-bitmap-filters-in-microsoft-sql-server.html

[20] Junyi Zhao, Kai Su, Yifei Yang, Xiangyao Yu, Paraschos Koutris, and Huanchen Zhang. 2025. Debunking the Myth of Join Ordering: Toward Robust SQL Analytics. *Proc. ACM Manag. Data* 3, 3 (2025), 146:1–146:28. doi:10.1145/3725283

[21] Rui Zhong, Yongheng Chen, Hong Hu, Hangfan Zhang, Wenke Lee, and Dinghao Wu. 2020. SQUIRREL: Testing Database Management Systems with Language Validity and Coverage Feedback. In *CCS '20: 2020 ACM SIGSAC Conference on Computer and Communications Security, Virtual Event, USA, November 9-13, 2020*, Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna (Eds.). ACM, 955–970. doi:10.1145/3372297.3417260


## A The Existence of a Join Tree for a Bushy Plan of an ACQ

We first generalize the TTJ left-deep plan from Section 2.1 to a left-deep plan where an internal node can be any join implementation.

*Definition A.1.* A left-deep plan is a binary tree where internal nodes are joins, and every right child `Rinner` and the left child `Router` of the leftmost iterator are leaves.

We assume that for a given plan of a query $Q$, the leaves of the plan are labeled from left to right in increasing order $\ell_1, \ldots, \ell_{|Q|}$. We iterate through the leaves from right to left and find the first *maximal* left-deep subplan. A left-deep subplan is maximal if adding any additional internal node and its associated children from the bushy plan prevents the resulting plan from being left-deep. We call a plan $P$ *terminal* if the first maximal left-deep subplan of $P$ is $P$ itself. Thus any left-deep plan is terminal.

For any left-deep subplan $q$ of $P$, we consider $q$ to be a query and hence $|q|$ is the number of relations in $q$. Let $\Sigma(R)$ produce the attribute set of relation $R$. Abusing a bit of notation, let $\Sigma(P)$ denote the union of attributes of relations that are leaves of plan $P$ and let $\Sigma(T)$ denote the union of attributes of all relations associated with nodes in a join tree $T$. Let $\mathrm{Par}(R, P)$ take in a relation $R$ and a plan $P$ and return the parent of the node associated with $R$ in plan $P$; let $\mathrm{Child}(R, P)$ take in a relation $R$ and a plan $P$ and return the right child of the node associated with $R$ in plan $P$; and let $\mathrm{Root}(P)$ return the root node of a plan $P$. When we label a leaf with a plan name as the superscript, the subscript of the leaf indicates the rank of the leaf within the plan indicated by the superscript name.

*Definition A.2.* Let $T_1 = (X_1, E_1)$ (resp., $T_2 = (X_2, E_2)$) be a tree where $X_1$ (resp., $X_2$) is a set of nodes and $E_1$ (resp., $E_2$) is a subset of $X_1 \times X_1$ (resp., $X_2 \times X_2$). Moreover, $X_1 \cap X_2 = \{u\}$ for some node $u$. We define $\mathrm{Mrg}(T_1, T_2)$ for $T_1$ and $T_2$ as follows. The output of $\mathrm{Mrg}(T_1, T_2)$ is a graph $T = (X, E)$ where $X = X_1 \cup X_2$ and $E = E_1 \cup E_2$. It is straightforward to see that $T$ is a tree.

*Definition A.3 (the reverse of a GYO reduction order [5]).* Let $P$ be a left-deep plan with leaves $\ell_1, \ldots, \ell_{|Q|}$. Plan $P$ satisfies the reverse of a GYO reduction order if for every $\ell_j$ ($2 \le j \le |Q|$), there exists some $\ell_i$ ($1 \le i < j$) such that $K \subseteq \Sigma(\ell_i)$ where $K$ is equal to $\Sigma(\ell_j) \cap \left(\bigcup_{r=1}^{j-1} \Sigma(\ell_r)\right)$.

*Definition A.4 (replacement).* We define a sequence of replacements applied towards a plan $P$ as follows. Let $P^i$ denote the input to the $i$th replacement ($i \ge 0$). Let $P^0$ be $P$. Replacement $i$ takes in the plan output of replacement $i-1$ and outputs another plan. (Replacement 0 takes in $P^0$.) Consider the $i$th replacement $\mathrm{Repl}^*(P^i)$. Let $q_i$ be the first maximal left-deep subplan of $P^i$. Let $V$ denote the virtual relation of $q_i$ where $\Sigma(V)$ is $\Sigma(q_i)$. The output plan $P^{i+1}$ is constructed as follows. We set $\mathrm{Child}(\mathrm{Par}(\mathrm{Root}(q_i), P), P)$ to be $V$. Then we discard all the nodes that appear in both $q_i$ and $P$ from $P$. The remaining $P$ is $P^{i+1}$. The replacement sequence stops after $\mathrm{Repl}^*(P^m)$ (for some $m > 0$) where $P^m$ is terminal.

From Definition A.4, we have the following observation.

Proposition A.5. *Each replacement produces one virtual relation and the virtual relation is a leaf node that is not the leftmost.*

Hence let $\mathrm{Virt}(P^{i+1})$ denote the lone virtual relation that is built from $\mathrm{Repl}^*(P^i)$.

*Definition A.6 (nice).* A bushy plan $P$ is *nice* if (1) $P$ does not have Cartesian products, (2) any left-deep subplan $q$ of $P$ satisfies the reverse of a GYO reduction order, and (3) for all $i \geq 0$, let $\ell_j$ denote $\mathrm{Virt}(P^{i+1})$ for some $j > 1$, there exists $1 \leq r < j$ such that $K \subseteq \Sigma(\ell_r)$ where $K$ is the key of $\ell_j$.

Lemma A.7. *Suppose a sequence of $m$ replacements applied to a nice bushy plan $P$: $\mathrm{Repl}^*(P^0), \ldots, \mathrm{Repl}^*(P^m)$. Then for all $0 \leq i \leq m$, the first maximal left-deep subplan $q_i$ of $P^i$ satisfies the reverse of a GYO reduction order.*

Proof. Let $0 \leq r \leq m$ and let $G(r)$ denote the predicate "the first maximal left-deep subplan of $P^r$ satisfies the reverse of a GYO reduction order." We prove by induction that $G(r)$ holds for all $r = 0, \ldots, m$.

Base case: By the left-deep plan definition and Definition A.6 (2), the first maximal left-deep subplan $q_0$ of $P^0$ satisfies the reverse of a GYO reduction order.

Induction step: Let $r$ belong to $\{1, \ldots, m\}$ and assume that $G(r')$ holds for all $r'$ in $\{0, \ldots, r-1\}$. We need to prove that $G(r)$ holds. Let $P^r$ be the output of $\mathrm{Repl}^*(P^{r-1})$. Consider the first maximal left-deep subplan $q_r$ of $P^r$ and a leaf $\ell_i^{q_r}$. Since $i = 1$ holds vacuously under Definition A.3, we consider two cases where $i > 1$.

Case 1: $\ell_i^{q_r}$ is a non-virtual relation. Leaf node $\ell_i^{q_r}$ is non-virtual in $P$. Suppose $\ell_i^{q_r}$ corresponds to $\ell_{i'}^P$ in $P$. By Definition A.6 (2) and Definition A.3, there exists a leaf $\ell_{j'}^P$ for some $1 \leq j' < i'$ such that the key of $\ell_{i'}^P$ is a subset of $\Sigma(\ell_{j'}^P)$ and $\ell_{i'}^P$ belongs to the same left-deep subplan as $\ell_{j'}^P$. Since $\ell_{i'}^P$ is not in $q_{r-1}$, and $1 \leq j' < i'$, $\ell_{j'}^P$ is not in $q_{r-1}$. Furthermore, since all the nodes in the first maximal left-deep subplan of $P^{r'}$ for all $r'$ in $\{0, \ldots, r-1\}$ are discarded per Definition A.4, $\ell_{j'}^P$ is not in $q_{r'}$. Thus $\ell_{j'}^P$ corresponds to $\ell_j^{q_r}$ for some $1 \leq j < i$. Hence $\Sigma(\ell_j^{q_r}) = \Sigma(\ell_{j'}^P)$, which contains the key of $\ell_i^{q_r}$.

Case 2: $\ell_i^{q_r}$ is a virtual relation. By Proposition A.5, we have $i > 1$. By Definition A.6 (3), there exists a preceding relation $\ell_j^{q_r}$ $(1 \leq j < i)$ whose attributes contain the key of $\ell_i^{q_r}$.

Combining the two cases, we conclude that $G(r)$ holds, as required. □

Lemma A.8. *If a left-deep plan $P$ satisfies the reverse of a GYO reduction order, then $P$ corresponds to a join tree with root $\ell_1^P$.*

Proof. Immediate from Lemma 4.6 of [5]. □

Lemma A.9. *Let $P$ be a left-deep plan with leaves $\ell_1, \ldots, \ell_{|P|}$ and let $V$ be a virtual relation where $\Sigma(V) = \Sigma(P)$. Then the sequence $S = \{V, \ell_1, \ldots, \ell_{|P|}\}$ satisfies the reverse of a GYO reduction order.*

Proof. Since $\Sigma(V)$ is $\Sigma(P)$, the key of $\ell_1$ is a subset of $\Sigma(V)$. Let $1 < i \leq |P|$. Since $\Sigma(V)$ is $\Sigma(P)$, the key of $\ell_i$ in $S$ is $\Sigma(\ell_i) \cap \left(\Sigma(V) \cup \bigcup_{r=1}^{i-1} \Sigma(\ell_r)\right) = \Sigma(\ell_i) \cap \Sigma(V)$, which is contained in $\Sigma(V)$. Since $i$ is chosen arbitrarily, we conclude that $S$ satisfies the reverse of a GYO reduction order, as required. □

*Definition A.10 (virtual relation method).* Let $P$ be a nice bushy plan. We enhance a sequence of replacements in Definition A.4 to construct a tree as follows. Let $P^i$ denote the input to the $i$th replacement $(i \geq 0)$. We extend $\mathrm{Repl}^*(P^i)$ with a tree $T_i$ as an additional input. The function produces a tree $T_{i+1}$ as an additional output. Thus we define a new function $\mathrm{Repl}(P^i, T_i)$, which produces a tuple of two elements $P^{i+1}$ and $T_{i+1}$. Let $T_0$ be an empty set.

Consider the $i$th replacement $\mathrm{Repl}(P^i, T_i)$. Let $q_i$ be the first maximal left-deep subplan of $P^i$. Let $V$ denote the virtual relation of $q_i$ where $\Sigma(V)$ is $\Sigma(q_i)$. The output plan $P^{i+1}$ is constructed the same as Definition A.4. Tree $T_{i+1}$ is constructed as follows. Let $q_i$ be the first maximal left-deep subplan of $P^i$. We define a function $\mathrm{Choose}(q_i) = \mathrm{Virt}(P^{i+1})$. By Lemmas A.8 and A.9, we construct a join tree $T^{q_i}$ with root $\mathrm{Choose}(q_i)$ via the method described in Section 4.2. Then $T_{i+1}$ is $\mathrm{Mrg}(T^{q_i}, T_i)$.

Proposition A.11. *Let plan $P^i$ and tree $T_i$ denote the output of $\mathrm{Repl}(P^{i-1}, T_{i-1})$ for some $i \geq 1$. Let $q_{i-1}$ denote the first maximal left-deep subplan of $P^{i-1}$. Then $\Sigma(\mathrm{Virt}(P^i))$ is equal to $\Sigma(q_{i-1})$, which is equal to $\Sigma(T_i)$.*

Proof. Immediate from Definition A.10. □

We remark that the virtual relation method does not actually modify $P$. Since the virtual relation method enhances Definition A.4 with tree construction, we have the following lemma.

Lemma A.12. *Suppose a sequence of $m$ replacements applied to a nice bushy plan $P$: $\mathrm{Repl}(P^0, T_0), \ldots, \mathrm{Repl}(P^m, T_m)$. Then for all $0 \leq i \leq m$, the first maximal left-deep subplan $q_i$ of $P^i$ satisfies the reverse of a GYO reduction order.*

For a tree $T$ where each node corresponds to a relation, we identify each node with its associated relation. For example, a relation is in $T$ if there exists a node in $T$ that is associated with the relation. We say an attribute $a$ appears in $T$ if there exists some relation $R$ in $T$ such that $a \in \Sigma(R)$. We say a relation $R$ contains attribute $a$ if $a \in \Sigma(R)$.

Lemma A.13. *Let $T_1 = (X_1, E_1)$ (resp., $T_2 = (X_2, E_2)$) be a join tree where $X_1$ (resp., $X_2$) is a set of nodes and $E_1$ (resp., $E_2$) is a subset of $X_1 \times X_1$ (resp., $X_2 \times X_2$). Moreover, $X_1 \cap X_2 = \{u\}$ for some node $u$ where the associated relation $R$ has $\Sigma(R) = \Sigma(T_2)$. Let $b$ be an attribute that appears in both $T_1$ and $T_2$. Relations in $\mathrm{Mrg}(T_1, T_2)$ that contain $b$ form a connected subtree.*

Proof. Let $T$ be $\mathrm{Mrg}(T_1, T_2)$. Since $T$ is a tree, it is sufficient to show that for any two relations in $T$, if the relations contain $b$, all the relations on the path between the two relations contain attribute $b$. Since $\Sigma(R) = \Sigma(T_2)$ and $b$ appears in $T_2$, $b$ is in $\Sigma(R)$. Let $S$ be a relation in $T_1$ that contains $b$. Since $T_1$ is a join tree, $S$ and $R$ belong to a connected subtree, which indicates that all the relations on the path from $S$ to $R$ contain $b$. Let $W$ be a relation in $T_2$ that contains $b$. By a similar argument as above, all the relations on the path between $R$ and $W$ contain $b$. Hence we conclude that all the relations on the path between $S$ and $W$ contain $b$. Since $S$ and $W$ are chosen arbitrarily, we conclude that the claim of the lemma holds. □

Lemma A.14 completes Section 4.3 and shows that with the help of the virtual relation method, it is possible to construct a join tree from nice bushy plans.

Lemma A.14. *Let $P$ be a nice bushy plan of an ACQ. Then the virtual relation method from Definition A.10 constructs a join tree from $P$.*

Proof. Suppose the sequence of replacements stops after the $m$th replacement (i.e., $\mathrm{Repl}(P^0, T_0), \ldots, \mathrm{Repl}(P^m, T_m)$). Let $1 \leq r \leq m+1$ and let $G(r)$ denote the predicate "$T_r$ from $\mathrm{Repl}(P^{r-1}, T_{r-1})$ is a join tree." We prove by induction that $G(r)$ holds for all $r = 1, \ldots, m+1$.

Base case: By the definition of left-deep plan, Definition A.6 (2), and Lemma A.8, tree $T_1$ from the first maximal left-deep subplan $q_0$ of $P^0$ is a join tree.

Induction step: Let $r$ belong to $\{2, \ldots, m+1\}$ and assume that $G(r')$ holds for all $r'$ in $\{1, \ldots, r-1\}$. We need to prove that $G(r)$ holds. Let $q_{r-1}$ be the first maximal left-deep subplan of $P^{r-1}$. By Lemma A.12, $q_{r-1}$ satisfies the reverse of a GYO reduction order. By Definition A.10, tree $T^{q_{r-1}}$ from $q_{r-1}$ is a join tree. Next we argue that $T_r$, the output of $\mathrm{Mrg}(T^{q_{r-1}}, T_{r-1})$, is a join tree by showing it satisfies RIP. By Proposition A.11, we have $\Sigma(\mathrm{Virt}(P^{r-1})) = \Sigma(T_{r-1})$. Consider an attribute $a$ that appears in some relation in $T_r$. If $a$ appears in $T_{r-1}$ but not in $T^{q_{r-1}}$, or appears in $T^{q_{r-1}}$ but not in $T_{r-1}$, RIP holds trivially since $T_{r-1}$ and $T^{q_{r-1}}$ are join trees. If $a$ appears in both $T_{r-1}$ and $T^{q_{r-1}}$, since $T^{q_{r-1}}$ and $T_{r-1}$ are join trees, $T^{q_{r-1}}$ and $T_{r-1}$ share exactly one node $\mathrm{Virt}(P^{r-1})$, and $\Sigma(\mathrm{Virt}(P^{r-1})) = \Sigma(T_{r-1})$, by Lemma A.13, we conclude that relations in $\mathrm{Mrg}(T^{q_{r-1}}, T_{r-1})$ that contain $a$ form a connected subtree. Combining all three cases, we conclude that $G(r)$ holds, as required. Since $P^m$ is terminal and $G(r)$ holds, we conclude that the claim of the lemma holds. □